Taumata Arowai

Coordinate-Based Zone Lookup Dashboard

Technical Design and Prototype Report

Prepared by: Team 06
Date: October 10, 2026

Contents

1 Executive Summary **1**

2 Introduction **2**
2.1 Background . . . 2
2.2 Problem Statement . . . 2
2.3 Objectives . . . 2
2.4 Scope . . . 2

3 Data and Geospatial Context **3**
3.1 Source Data and Layers . . . 3
3.2 Observed Attribute Schema . . . 3
3.3 Coordinate Reference System . . . 3

4 System Design **5**
4.1 High-Level Workflow . . . 5
4.2 Technology Stack . . . 5
4.3 Why Streamlit . . . 5

5 User Interface and Prototype **7**
5.1 Interface Behaviour . . . 7
5.2 Screenshot: Main Interface . . . 7
5.3 Screenshot: Successful Lookup . . . 8
5.4 Data Privacy . . . 8

6 Validation, Risks, and Limitations **9**
6.1 Validation Plan . . . 9
6.2 Known Limitations . . . 9
6.3 Risk Controls . . . 9

7 Future Development Plan **10**
7.1 Phase 1: Validate the Spatial Layer . . . 10
7.2 Phase 2: Add Geographic Coordinates . . . 10
7.3 Phase 3: Add Textual Address-to-Coordinate Lookup . . . 10
7.4 Phase 4: Map and Result Improvements . . . 11
7.5 Phase 5: Deployment and Integration . . . 11

8 Conclusion **12**

1 Executive Summary

This project explores a user-facing dashboard for identifying the geographic zone associated with a location in New Zealand. The intended longer-term workflow is for a user to enter a street address and receive the relevant zone or network information. The initial prototype deliberately accepts coordinates so that spatial matching can be tested independently of address geocoding.

The supplied data was received in LPRX format and extracted for inspection in QGIS as a geodatabase with the `.gdb` extension. The inspected layer containing divided geographic sections reports EPSG:3857 (WGS 84 / Pseudo-Mercator). Its observed fields include `OBJECTID`, `zone`, population-related fields, `Shape_Length`, and `Shape_Area`.

A prototype has been prepared using Python, Streamlit, GeoPandas, and Shapely. It accepts projected X and Y coordinates, transforms the input point to the polygon layer CRS if needed, and tests which polygon or polygons cover the point. It displays matching attributes and warns when there are no matches or multiple matches.

This is a proof of concept, not yet a production service. The semantic meaning of the selected layer must be confirmed against authoritative metadata before results are described as water supply network assignments. Future work includes spatial validation, boundary handling, a map, geographic-coordinate input, and an approved address-to-coordinate component.

2 Introduction

2.1 Background

The project concerns a dashboard for a Taumata Arowai-related use case in New Zealand. The intended user journey is to enter an address and identify the corresponding zone or network from the supplied spatial data. QGIS is being used to inspect the data and test the spatial logic before integration into a user interface.

2.2 Problem Statement

The user should be able to provide a location and receive a clear, reproducible lookup result. The supplied data contains multiple layers, including one layer with divided polygonal sections and other layers that appear to be distance buffers around the coastline. The correct meaning of each layer must be established before production use.

The first milestone is: given a coordinate pair, identify the polygon feature or features that cover that point and return their relevant attributes. Address parsing and geocoding are outside the first prototype's scope.

2.3 Objectives

1. Inspect the geodatabase and identify the candidate zone layer.
2. Establish a repeatable point-in-polygon query.
3. Provide a simple interface for coordinate entry and result display.
4. Make coordinate reference system (CRS) handling explicit.
5. Document limitations and a staged plan for address-based lookup.
6. Keep the implementation modular enough to support future integration.

2.4 Scope

Included: manual X/Y input in EPSG:3857; loading a polygon layer from a local geodatabase; CRS transformation when needed; point-in-polygon matching; display of attributes; and no-match/multiple-match warnings.

Not yet included: textual address search; a production map interface; automatic resolution of overlapping/boundary matches; hosting, authentication, monitoring, or operational support; and formal confirmation that the selected layer is the authoritative water supply zone/network layer.

3 Data and Geospatial Context

3.1 Source Data and Layers

The supplied data was received in LPRX format and extracted into a geodatabase for inspection in QGIS. A File Geodatabase is a container for spatial layers and tables. The divided polygon layer is the current candidate for the spatial lookup. Other layers appear to be coastal distance buffers and should not be used as service zones unless their intended meaning is confirmed.

A layer's appearance alone does not establish its business meaning. Population and deprivation-related field names could indicate demographic/statistical geography. Confirm the layer's purpose against source metadata, documentation, and project requirements before presenting results as water network assignments.

3.2 Observed Attribute Schema

Field	Type
OBJECTID	Integer64
zone	String
pop_regist	Real
pop_low_de	Real
pop_mod_de	Real
pop_high_d	Real
pop_maori	String
Shape_Length	Real
Shape_Area	Real

3.3 Coordinate Reference System

The inspected layer reports:

- CRS: EPSG:3857
- Name: WGS 84 / Pseudo-Mercator
- Coordinates: projected X and Y map coordinates, generally expressed in metres.

EPSG:3857 coordinates are not longitude and latitude. X is the horizontal projected coordinate and Y the vertical projected coordinate. The observed extent was:

```
X min:  19416362.0735
Y min:  -4476361.6471
X max:  19482457.0019
Y max:  -4335393.6076
```

This is the layer's rectangular bounding extent, not the boundary of a single zone. It is consistent with an area around Auckland, but should be interpreted through a verified CRS transformation. A reported accuracy of approximately 2 metres should be checked in source metadata: a CRS does not itself guarantee source-data positional accuracy. EPSG:3857 is commonly used for web mapping, but its scale distortion means raw lengths and areas should not automatically be treated as accurate ground measurements.

4 System Design

4.1 High-Level Workflow

1. User enters X and Y coordinates in EPSG:3857.
2. Application loads the configured polygon layer from the geodatabase.
3. Application creates a point and assigns its input CRS.
4. If needed, the point is transformed to the polygon layer CRS.
5. The application finds valid polygons that cover the point.
6. Matching attributes are displayed.
7. A message is shown for zero or multiple matches.

4.2 Technology Stack

Technology	Role
QGIS	Inspect layers, attributes, CRS, and test points.
Python	Application and lookup logic.
Streamlit	Browser-based user interface.
GeoPandas	Read spatial data and manage geometries/CRS.
Shapely	Point-in-polygon geometry operation.
File Geodatabase (`.gdb`)	Local source container for layers and attributes.

4.3 Why Streamlit

Streamlit was selected for the initial prototype because the main task is an interactive spatial lookup driven by Python code.

- **Direct Python integration:** the same environment can load data, transform coordinates, run geometry predicates, and display results.
- **Rapid prototyping:** input fields, buttons, tables, validation messages, and maps can be built with relatively little interface code.
- **Control of spatial behaviour:** zero, one, and multiple matches can be handled explicitly and tested.
- **Modularity:** the lookup logic can later be exposed as a reusable function or service for another frontend.

Power BI remains a viable choice for reporting and analytics, especially where it is already part of the organisation's workflow. Robust address-to-polygon matching may nevertheless need to occur upstream in Python or a spatial database, with Power BI consuming prepared results or a service. The final choice should consider hosting, licensing, accessibility, governance, and operational requirements.

5 User Interface and Prototype

5.1 Interface Behaviour

The interface accepts X and Y coordinates and runs the lookup when the user selects the search button. It can display `OBJECTID`, `zone`, and available population-related attributes; show a no-match warning; report multiple matches; and display setup or processing errors. It is a local prototype, not yet a production-hosted service.

5.2 Screenshot: Main Interface

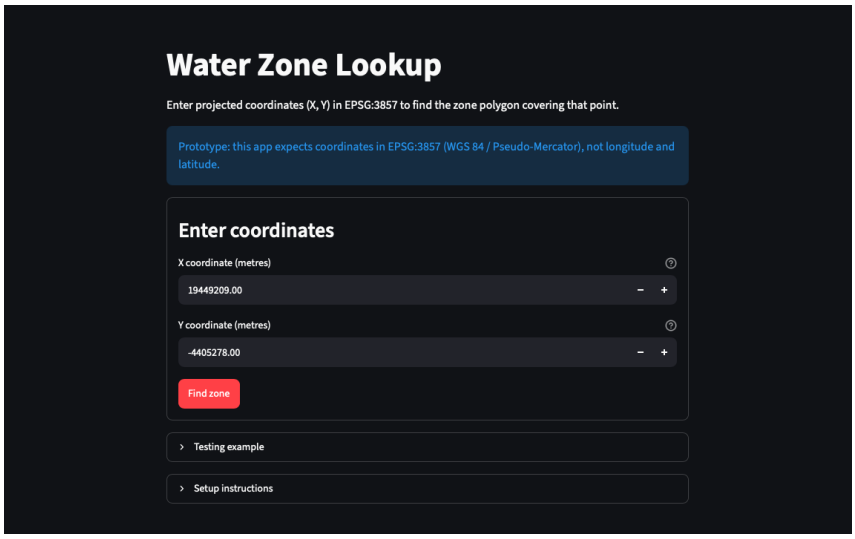


Figure 5.1: Main coordinate-input interface. Screenshot to be added.

5.3 Screenshot: Successful Lookup

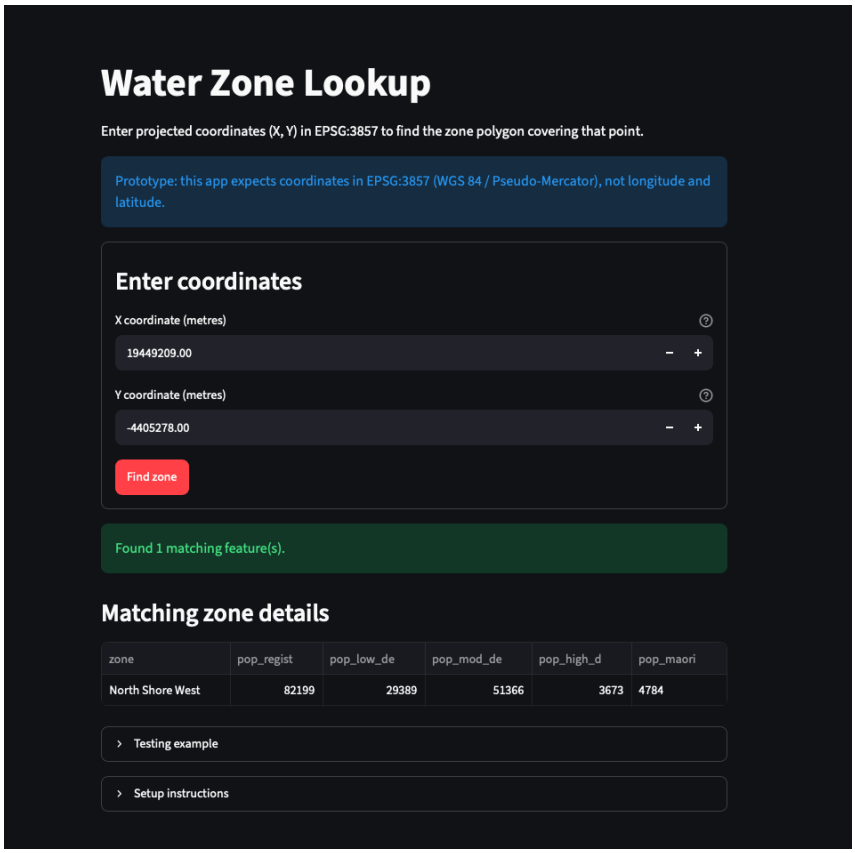


Figure 5.2: Successful coordinate-to-zone lookup. Screenshot to be added.

5.4 Data Privacy

The geodatabase is not bundled with the source code. Keep it in an approved location and do not commit it to a public repository or unapproved hosting service. Any hosted deployment should be reviewed against organisational data governance and security requirements.

6 Validation, Risks, and Limitations

6.1 Validation Plan

Test case	Purpose	Expected behaviour
Point inside a polygon	Verify basic lookup	Return expected feature.
Point outside polygons	Check coverage	Clear no-match message.
Point on shared boundary	Check edge behaviour	Report all matches or apply a documented rule.
Point in an overlap	Check overlapping geometries	Report multiple matches.
Invalid coordinates	Check input validation	Reject invalid input clearly.
Input in a different CRS	Check transformation	Transform correctly or reject unlabelled CRS.

6.2 Known Limitations

- The zone layer's authoritative business meaning has not yet been confirmed.
- Textual address lookup is not implemented.
- The current interface expects EPSG:3857 coordinates.
- Boundary or overlapping polygons can produce multiple matches.
- Geometry validity and coverage should be checked.
- Performance must be assessed using the actual dataset and deployment environment.
- The current version is a local prototype.

6.3 Risk Controls

Confirm the intended meaning of the layer before reporting a water supply assignment; validate representative points against QGIS and authoritative references; investigate no-match and multiple-match cases; document dataset versions and preprocessing; handle CRS explicitly; and apply appropriate access control for any deployment.

7 Future Development Plan

7.1 Phase 1: Validate the Spatial Layer

1. Confirm layer provenance, intended meaning, and authoritative status.
2. Determine whether `zone` is a stable identifier, label, or repeated category.
3. Check geometry validity, null geometries, overlaps, gaps, and feature counts.
4. Compare test coordinates against QGIS and document expected results.
5. Agree on a rule for multiple matches.

7.2 Phase 2: Add Geographic Coordinates

The interface can support longitude and latitude in EPSG:4326 as an alternative to projected X/Y in EPSG:3857. Clearly label the input mode and validate plausible ranges. Create the point in EPSG:4326 and transform it to the polygon layer CRS before querying:

```
point_4326 = gpd.GeoDataFrame(
    geometry=[Point(longitude, latitude)],
    crs="EPSG:4326"
)
point_for_query = point_4326.to_crs(zones.crs)
point = point_for_query.geometry.iloc[0]
matches = zones[zones.geometry.covers(point)]
```

This conversion changes coordinate representation, not the geographic location.

7.3 Phase 3: Add Textual Address-to-Coordinate Lookup

The final user journey starts with a New Zealand address. This requires a separate address-resolution/geocoding component:

1. User types an address.
2. An approved New Zealand address dataset or authorised geocoding service searches for candidates.
3. If multiple candidates are found, the user selects the intended address.
4. The selected address is returned as coordinates, usually longitude/latitude in EPSG:4326.
5. The point is transformed to the zone layer CRS.
6. The existing spatial lookup returns the polygon attributes.

Do not treat address-to-coordinate conversion as an unverified guess. Assess coverage, update frequency, licensing, privacy, rate limits, and suitability for public-service use. An authoritative address-point dataset may be preferable where available and permitted.

7.4 Phase 4: Map and Result Improvements

Potential improvements include an interactive map showing the point and polygon, zooming to the matched zone, presenting user-friendly labels, explaining ambiguous/out-of-coverage results, and providing a copyable result summary.

7.5 Phase 5: Deployment and Integration

Streamlit may remain suitable if it meets user and operational requirements. Alternatively, separate the spatial lookup into a reusable function or API. A future architecture could combine a frontend, a backend geocoding/spatial service, and a spatial database such as PostGIS if scale, concurrency, auditing, or central data management justify it. Power BI may be used for supplementary reporting and analytics. Production planning should cover hosting, authentication, availability, access control, logging, data updates, and service ownership.

8 Conclusion

A first coordinate-based zone lookup interface has been prepared using Python, Streamlit, GeoPandas, and Shapely. Separating coordinate-based spatial matching from address resolution provides a practical way to validate the geospatial logic with known points.

The next priority is data validation: confirm that the selected layer represents the intended service zones, verify the field definitions, and test ordinary, boundary, overlapping, and out-of-coverage locations. Geographic-coordinate input and an approved address-to-coordinate component can then be added without replacing the core lookup operation.

Streamlit suits a Python-centred prototype and may support a lightweight application. Power BI may be appropriate for reporting and analytics. The production choice should follow requirements gathering, validation, security review, and user testing.